ХАКАТОН «ЛИДЕРЫ ЦИФРОВОЙ ТРАНСФОРМАЦИИ» 2026 · КЕЙС №8
ДЕПАРТАМЕНТ ЖКХ ГОРОДА МОСКВЫ / АО «МОСКОЛЛЕКТОР»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА «МОСКОЛЛЕКТОР.НЕЙРОКОНТУР»

Сервис прогнозирования инцидентов и управления ремонтными работами инженерных коллекторов Москвы.
Редакция для финального этапа (октябрь 2026).

Параметр	Значение
Заказчик задачи	АО «Москоллектор», Департамент жилищно-коммунального хозяйства города Москвы
Команда	Vector
Объект автоматизации	Коллекторы Москвы (825 км); в данных организаторов — 11 485 каналов СМВУ
Что делает решение	Ранжирует каналы по риску инцидента на горизонте 24–72 ч; ловит быстрые опасные события (газ, перегрев, пожар, затопление, дребезг, потеря связи); ведёт журнал решений диспетчера; готовит черновики заявок ТО/ППР
Стек	Python 3.10, FastAPI, LightGBM, scikit-learn, Pydantic v2; React 18, TypeScript, Vite, Leaflet; Docker
Состояние	Рабочий прототип: REST API, веб-интерфейс двух ролей, 108 автотестов; подключения к СМВУ и CMMS заказчика нет (интеграция описана в п. 9)
Исходный код и стенд	SourceCraft (основной репозиторий), GitHub (зеркало), стенд на Render; ссылки — в README.md

[КАК ЧИТАТЬ ЦИФРЫ] Все результаты проверки прогноза получены на журнале СМВУ за 2026 год, предоставленном организаторами. Актов ремонтов (1С:ТОИР / CMMS) в данных нет, поэтому «событие» — это разметка по журналу (неисправность, сбой связи, сброс часов, опасное значение), а не подтверждённая авария. Результаты Сценарного полигона получены на синтетических данных и проверяют логику, а не качество на реальных авариях.

1. Назначение и подход

Задача из технического задания: заранее предупреждать диспетчера о риске инцидента в коллекторах, показывать вероятность, горизонт, локацию и причины, фиксировать решение диспетчера в журнале и готовить рекомендации и заявки на обслуживание. Интеграция с внутренними системами заказчика допускается только в режиме «только чтение».

Мы разделили задачу на два слоя, потому что у них разная скорость и разный способ проверки:

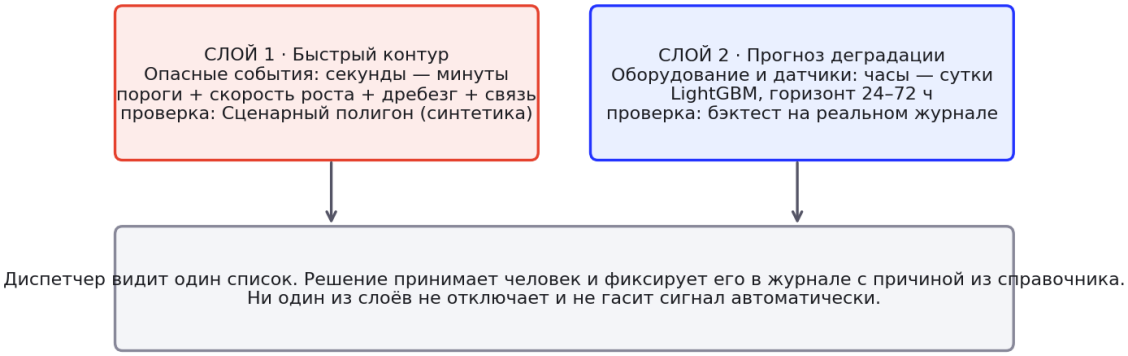


Рис. 1. Два слоя решения и общий список диспетчера

- **Слой 1 — быстрый контур:** правила по порогам и скорости роста значений ловят газ, перегрев, пожар, затопление, дребезг контактов и потерю связи за секунды и минуты. Проверяются на Сценарном полигоне (п. 5); на реальных данных такие события в журнале практически не встречаются.
- **Слой 2 — прогноз деградации:** модель LightGBM оценивает по истории канала, насколько вероятна неисправность или опасное значение в окне 24–72 часа. Проверяется бэктестом на реальном журнале (п. 4).

Что реализовано и что остаётся на пилот

Возможность	В прототипе	На пилоте у заказчика
Прогноз риска по каналам на 24–72 ч, объяснение факторов	да, на журнале организаторов	подключение потока СМВУ, дообучение на актах ТОИР
Быстрый контур: газ, температура, пожар, затопление, дребезг, потеря связи	да, на синтетических сценариях	настройка порогов по паспортам оборудования
Журнал решений диспетчера с цепочкой SHA-256	да	перенос в СУБД заказчика (PostgreSQL)
Заявки ТО/ППР (черновики)	да, JSON-хранилище	выгрузка в CMMS / 1С:ТОИР
Две роли: техник («Поле») и диспетчер («Сводка», «Обстановка»)	да	вход через каталог пользователей (LDAP/AD)
Режим «только чтение» к данным заказчика	да (прототип читает файлы)	read-only учётная запись СМВУ
Подключение к СМВУ / ОДС / CMMS в реальном времени	нет (описано в п. 9)	основная работа пилота
Еженедельное дообучение модели	проверено бэктестом; в демо стоит январская модель	автоматизировать по расписанию

2. Функциональная и компонентная архитектура

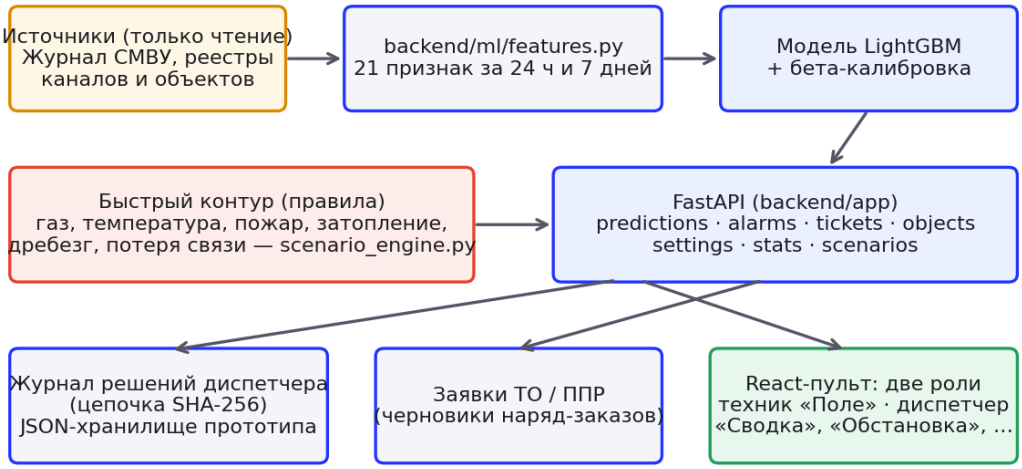


Рис. 2. Компоненты решения и потоки данных

Решение — отдельный сервис (REST API + веб-интерфейс), который можно развернуть самостоятельно или встроить как компонент в существующую систему. Исходный код открыт, не обфусцирован; сложные места снабжены комментариями.

Компонент	Где в репозитории	Назначение
Признаки	backend/ml/features.py	21 признак по каналу за 24 ч и 7 дней, правило разметки события
Обучение и метрики	backend/ml/train_validated_model.py	обучение LightGBM, калибровка, сравнение с логистической регрессией и случайным лесом
Бэктест	scripts/rolling_backtest.py	21 недельная проверка вперёд по времени, отчёт backend/models/rolling_backtest_report.json
API	backend/app/api/*.py	predictions, alarms, tickets, objects, settings, stats, simulation, scenarios, synthetic
Сервисы	backend/app/services/*.py	ml_service (скоринг, журнал решений, PIN), data_service (реестры), scenario_engine (быстрый контур и полигон)
Интерфейс	frontend/src/*	shell (вход и роли), field (техник), summary и situation (диспетчер), scenarios, components
Тесты	tests/*.py	108 автотестов: API, калибровка, бэктест, синтетический пайплайн, Сценарный полигон

Программные интерфейсы (REST)

Полная схема — OpenAPI: страница /docs работающего сервиса и файл /api/openapi.json. Основные группы:

Метод и путь	Назначение
GET /api/predictions · GET /api/predictions/{channel_id}	список и карточка прогноза по каналам (вероятность, уровень, факторы, рекомендуемое действие)
POST /api/predictions/score	онлайн-оценка вектора признаков датчика
GET /api/predictions/backtest · /metrics · /benchmark · /reconciliation	отчёты проверки: недельный бэктест, метрики модели, замеры скорости, сверка с эталоном
POST /api/alarms/classify · POST /api/alarms/confirm · GET /api/alarms/confirmed	разбор сигнала (настоящая тревога или ложная), решение диспетчера с причиной, журнал
POST /api/alarms/session · GET /api/alarms/audit/verify	вход диспетчера по табельному номеру и PIN; проверка целостности журнала
GET /api/tickets · POST /api/tickets/generate · PATCH /api/tickets/{id}/status	заявки ТО/ППР: список, черновик, смена статуса
GET /api/objects · /api/objects/{id} · /api/objects/geojson/network	объекты, карточка, схема сети
GET и POST /api/settings · POST /api/settings/reset	настраиваемые параметры (п. 7)
GET /api/scenarios/catalog · POST /api/scenarios/run · POST /api/scenarios/series	Сценарный полигон: каталог, прогон, ряд канала
GET /api/alarms/recent · /dispatchers · POST /api/simulation/step · /api/synthetic/evaluate	последние события, справочник диспетчеров, шаг имитации, проверка на синтетическом наборе
GET /api/stats/summary · GET /api/health · GET /api/info	сводка, проверка живости, сведения о сервисе

Входные данные проверяются схемами Pydantic до любых изменений состояния: неверное значение даёт ответ 422, неизвестный канал — 404, в журнал и заявки при этом ничего не пишется.

3. Данные и методы обработки

Исходные данные

Журнал срабатываний СМВУ за январь–июнь 2026 года (около 30,7 млн записей: идентификатор канала, тип, значение, время), реестр каналов с тегом в дереве объектов и реестр типов каналов. Журнал в репозиторий не входит (1,5 ГБ, файл организаторов); его контрольная сумма SHA-256 записана в docs/DATA_PROVENANCE_AND_REPRODUCIBILITY.md. Кэш признаков в репозитории позволяет воспроизвести все метрики без исходного журнала.

Что считается событием (разметка)

В данных нет актов ремонтов, поэтому событие размечено правилом по журналу (backend/ml/features.py, is_failure_value): состояния «неисправен», «отключено устройство», «ошибка связи», «сбой питания», «обрыв датчика», «короткое замыкание», «нет данных» и др.; сброс часов контроллера на 01.01.1970; значение метана ≥ 5 %; температура ≥ 45 °C или ≤ −10 °C; напряжение питания вне 9–30 В. Метка канала = 1, если в окне +24…+72 часа от момента прогноза есть хотя бы одна такая строка.

> **[ОГРАНИЧЕНИЕ]** Это разметка по журналу, а не подтверждённые отказы оборудования. Все метрики в записке описывают совпадение с этой разметкой и не являются оценкой вероятности физической аварии, предотвращённых ремонтов или бухгалтерской экономии.

Подготовка данных

- Сброс часов на 1970 год считается признаком деградации питания контроллера, а не ошибкой данных: такие строки не удаляются, а учитываются отдельным признаком.

- Дребезг контактов (быстрые переключения одного дискретного канала) учитывается как признак и отдельно фиксируется быстрым контуром; автоматически сигнал не гасится.
- Для обучения и проверки признаки строятся только по данным до момента прогноза, окно метки лежит строго после него.

Признаки и модель

Группа	Признаки (21)
Активность канала	cnt_24h, cnt_7d, acc_events
Тревоги	alarms_24h, alarms_7d, alarm_ratio, acc_alarms
Дребезг и разнообразие состояний	chatter_cnt, chatter_ratio, unique_states
Связь и питание	silence_hours (молчание канала), battery_glitches, date_corruptions (сбросы часов)
Значения	num_mean, num_std, num_max, gas_spikes, temp_spikes
Контекст	sensor_type_code, system_type_code, obj_level (уровень объекта в иерархии)

Модель — градиентный бустинг LightGBM; вероятность калибруется бета-калибровкой, подобранной только на валидационной выборке. Порог очереди диспетчера настраивается (по умолчанию 0,42 по безразмерному баллу модели, это не вероятность). Для сравнения обучены логистическая регрессия и случайный лес. Обучение: backend/ml/train_validated_model.py, воспроизведение: scripts/reproduce_metrics.py.

4. Проверка прогноза на реальных данных (бэктест)

Проверка идёт «вперёд по времени», как в реальной работе: каждую неделю модель видит только прошлое и предсказывает окно +24…+72 часа после момента прогноза (00:00 дня отсечки, раз в неделю). Выполнено 21 еженедельных проверок (с 2026-02-04 по 2026-06-24), по 11 485 каналам в каждой. Воспроизведение: python scripts/rolling_backtest.py; отчёт — backend/models/rolling_backtest_report.json.

Что сравнивается

Вариант	Что это
deployed (в демо)	модель, которая стоит в прототипе: обучена один раз на январских данных и дальше не переобучалась
lgbm_weekly	та же архитектура LightGBM, но переобучаемая каждую неделю на всей накопленной истории (как будет на пилоте)
logreg_weekly	логистическая регрессия, переобучаемая еженедельно (простая линейная база)
persistence (правило)	«канал, у которого были неисправности за последние 7 дней, будет неисправен и дальше» — простое правило без обучения

Все варианты проверены на одних и тех же неделях и каналах. Метрики — по каждой неделе отдельно, ниже приведена медиана по 21 неделям.

Результаты: все каналы реестра

Вариант	ROC-AUC	PR-AUC	Точность в топ-50	в топ-100	в топ-200
deployed (в демо)	0,648	0,142	0,34	0,22	0,12
lgbm_weekly	0,845	0,274	0,56	0,38	0,24
logreg_weekly	0,792	0,197	0,48	0,29	0,20
правило persistence	0,704	0,181	0,48	0,38	0,28

Читать так: если диспетчер проверяет 100 каналов с наибольшим риском, то в среднем 22 % из них действительно окажутся с событием (демо-модель) или 38 % (модель с еженедельным переобучением). Доля событий среди всех каналов (базовая частота) — менее 1 %.

Сколько недель из 21 модель лучше правила

Показатель	deployed vs правило	lgbm_weekly vs правило
ROC-AUC (выиграла / проиграла / равно)	5 / 16 / 0	18 / 3 / 0
PR-AUC	6 / 15 / 0	18 / 3 / 0
Точность в топ-100	2 / 18 / 1	8 / 12 / 1
Точность в топ-200	1 / 20 / 0	12 / 9 / 0

Результаты: каналы без неисправностей за предыдущие 7 дней («новые инциденты»)

Правило persistence не видит такие каналы вообще — для него это слепая зона. Именно здесь модели нужна, чтобы дать то, чего правило дать не может.

Вариант	ROC-AUC	PR-AUC	Точность в топ-100
deployed (в демо)	0,578	0,022	0,05
lgbm_weekly	0,757	0,049	0,06
logreg_weekly	0,662	0,016	0,01
правило persistence	0,500	0,010	0,00

Для правила ROC-AUC = 0,5 по построению (оно не отличает один такой канал от другого).

[ЧТО ЭТО ЗНАЧИТ] Честный вывод. Модель, стоящая в демо (обучена в январе и не переобучалась), слабее простого правила: правило лучше или равно в большинстве недель по ROC-AUC и по точности в топе. Модель с еженедельным переобучением заметно лучше по ROC-AUC и PR-AUC, на топ-100 примерно равна правилу, на топ-200 уступает. Практическая ценность модели — в каналах, где правило слепо: события в таких каналах составляют 59 % всех событий за период (2 393 из 4 027), именно они определяют, сколько проблем выявляется заранее. Поэтому на пилоте мы закладываем еженедельное переобучение и сравнение с правилом на актах ТОИР, а не обещаем цифры демо.

5. Сценарный полигон: проверка быстрого контура

На реальных данных быстрых опасных событий почти нет, поэтому быстрый контур проверяется на синтетическом полигоне: 185 каналов, эмулируются сутки (шаг 60 с) и месяц (шаг 300 с) с

планово заложенными авариями. Спецификация с порогами и протоколом зафиксирована до первого прогона: docs/SCENARIOS_SPEC.md. Результаты ниже — на отложенных наборах (seed 2000–2019), которые прогнаны один раз после заморозки логики; файл docs/SCENARIOS_HELDOUT_RESULTS.json содержит версии кода и контрольные суммы.

> **[ВАЖНО]** Данные синтетические. Полигон проверяет, что логика срабатывает правильно и не создаёт лишнего шума. Это не оценка качества на реальных авариях Москоллектора.

Сценарий	Быстрый контур: найдено	Показатель	Медиана	Статич. порог: найдено	Медиана
S1 Медленная утечка газа	80 / 80	опережение	6,9 мин	80 / 80	-0,5 мин
S2 Быстрая утечка газа	60 / 60	задержка	62,2 с	60 / 60	62,2 с
S3 Перегрев трубопровода	80 / 80	опережение	37,2 мин	80 / 80	-0,4 мин
S4 Пожар (тепловой датчик)	60 / 60	задержка	0,0 с	60 / 60	0,0 с
S5 Затопление	60 / 60	задержка	0,0 с	60 / 60	0,0 с
S6 Дребезг контакта	200 / 200	задержка	17,2 с	200 / 200	2,9 с
S7 Потеря связи	80 / 80	задержка	928,2 с	0 / 80	не обнаружено
S8 Колебание газа у уставки	60 / 60	задержка	75,3 с	60 / 60	75,3 с

«Опережение» — на сколько минут раньше фиксированного порога 1 % метана (или 45 °C) мы сообщаем диспетчеру; положительное значение — раньше. «Задержка» — сколько секунд проходит от начала события до оповещения. Медианы по 20 наборам.

Сводка по детекторам (20 наборов × сутки)

Показатель	Быстрый контур	Статический порог + дедупликация	Наивный порог
События найдены	680 из 680	600 из 680	600 из 680
Ложные тревоги (всего)	1	80	80
Оповещений в сутки (в среднем)	146	535	1414
Инцидентов в списке диспетчера (в среднем)	67	502	1414

Нагрузка на диспетчера: наивный порог даёт в 21,1 раза больше оповещений, чем быстрый контур; статический порог с дедупликацией — в 7,5 раза больше инцидентов в списке.

Колебание у уставки (замечание экспертов)

Сценарий S8 моделирует газ, который «гуляет» около порога 1 %: статический порог без гистерезиса превращает его в десятки оповещений. Быстрый контур выдаёт 3 оповещения на событие (газовое предупреждение, рост газа и пометка «колебание у уставки») и помечает инцидент как колебание у уставки (100 % случаев); наивный порог даёт 15 оповещений на событие.

Ограничения полигона

- Статические базовые варианты получают ложные тревоги от заранее заложенных «нормальных» отклонений (отрицательные контрольные случаи): их 80 — следствие построения теста, а не независимое измерение.
- Для S7 (потеря связи) у статических вариантов нет правила вовсе — преимущество быстрого контура 680 из 680 против 600 из 680 по сути определяется этим сценарием.
- В режиме «месяц» шаг замеров 5 минут: опережение по S1 отрицательное (-2,6 мин по seed 42), пометка «колебание у уставки» ставится только в 12,5 % случаев. Для пилота нужна частота опроса газа не реже 1 раза в минуту.
- Пороги не подбирались по результатам отложенных наборов; пороги получены из требований заказчика и экспертов и из допущений команды (п. 7).

6. Скорость работы

Замер	Результат	Что измерено
Пакетная оценка всех 11 485 каналов	62,9 мс (p95 70,7 мс)	только расчёт модели на подготовленных признаках
Оценка одного канала	1,50 мс (p95 2,58 мс)	только расчёт модели на подготовленных признаках
HTTP-нагрузка: 500 запросов, 20 параллельных	111,3 запросов/с, медиана 172 мс, p95 252 мс	полный путь запроса к API через локальную сеть, без ошибок (100 %)

Требование ТЗ — не более 300 секунд на обработку. Замер пакетной оценки не включает чтение потока СМВУ и построение признаков из «сырых» записей: время построения признаков зависит от способа подключения к потоку и подлежит измерению на пилоте (этап «теневой режим»).

7. Пороги и настраиваемые параметры

Все пороги вынесены в настройки и перечислены в спецификации; в таблице указан источник каждого значения: [З] — задано заказчиком или экспертами, [К] — значение, уже стоявшее в коде, [Д] — допущение команды (будет уточняться на пилоте).

Параметр	Значение	Источник
Метан: предупреждение / тревога	1,0 % / 5,0 % объёмных	[З]
Рост метана	≥ 0,3 % за 3 замера и наклон ≥ 0,05 %/мин на окне max(5 мин; 3 шага)	[Д]
Температура: предупреждение / тревога	40 / 45 °C	[К]
Быстрый рост температуры	≥ 28 °C и наклон ≥ 0,2 °C/мин	[Д]
Дребезг контакта	≥ 5 переключений за 90 с; закрытие через 15 мин без новых	[К]/[Д]
Колебание у уставки газа	≥ 4 пересечения 1,0 % за 15 мин; сброс через 30 мин	[Д] по замечанию экспертов
Потеря связи	нет замеров дольше max(15 мин; 3 шага)	[Д]
Пожар	сработка теплового датчика — немедленно, с эскалацией по росту температуры	[З]
Затопление	датчик в тревоге — немедленно	[З]
Напоминание о неподтверждённой тревоге	каждые 30 мин	[Д]
Порог очереди диспетчера (модель)	0,42 по баллу модели (не вероятность)	[К]
Подтверждение диспетчером	обязательно (require_dispatcher_confirmation = true)	[З]
Автоподавление дребезга	выключено (auto_suppress_chatter = false)	[З]
Стоимости для расчёта эффекта	аварийный выезд 18 500 ₽; плановый осмотр 3 200 ₽	[Д] нормативные ставки, уточняются

Параметры меняются через экран «Настройки» или POST /api/settings и возвращаются к значениям по умолчанию через POST /api/settings/reset. Прогон Сценарного полигона использует зафиксированный снимок настроек и не зависит от того, что изменено на экране.

Тревога подтверждается или отклоняется диспетчером с указанием причины. Дребезг не гасится автоматически: его решает человек. Это сознательный выбор: ложное гашение при реальной аварии опаснее лишнего оповещения.

8. Безопасность и журнал решений

- Решения диспетчера (подтверждение/отклонение тревоги, причина, время, табельный номер) записываются в журнал confirmed_alarms.json; каждая запись содержит SHA-256 от своих полей (канал, решение, табельный номер, время, примечание) и хэша предыдущей записи. Проверка целостности — GET /api/alarms/audit/verify: любое изменение или удаление записи в середине цепочки обнаруживается.
- Вход диспетчера — табельный номер и PIN (в интерфейсе хранятся только в памяти вкладки); в хранилище лежит лишь хэш PIN (PBKDF2-SHA256), сравнение выполняется за постоянное время. Демонстрационные учётные данные (ДИСП-0001 / PIN из переменной окружения LCT_DEMO_DISPATCHER_PIN) предназначены только для стенда; на пилоте заменяются каталогом пользователей заказчика (LDAP/AD).
- Все входные данные проверяются схемами Pydantic; ответ 422 на неверные значения, запись в журнал и заявки при этом не выполняется.

- Прототип работает в режиме «только чтение» по отношению к данным заказчика: изменяются только собственные журналы решений и заявок прототипа.
- Контейнер запускается под непривилегированным пользователем (uid 1000), прокси-заголовки принимаются штатно.

Требования КИИ (187-ФЗ, приказы ФСТЭК № 235 и № 239), сетевая изоляция технологического сегмента через диод данных/DMZ, двухфакторная аутентификация и передача событий в SIEM описаны в docs/DATA_REQUEST_AND_PILOT_METHODOLOGY.md, раздел 5; в прототипе они не реализованы.

9. Интеграция с системами заказчика

В прототипе источником данных служат файлы организаторов. Для пилота предлагаются однонаправленные адаптеры «только чтение»; конкретный вариант выбирается заказчиком:

Способ	Что подключается	Статус
REST (HTTP/JSON)	получение прогнозов и тревог внешними системами; приём замеров POST /api/predictions/score	реализовано в прототипе
Kafka / MQTT / RabbitMQ	подписка на поток замеров СМВУ (read-only), запись результатов в отдельный топик	описано, адаптер на пилоте
SOAP / ESB предприятия	обмен с корпоративной шиной, выгрузка заявок в CMMS / 1С:ТОИР	описано, адаптер на пилоте
Файловый обмен (CSV)	выгрузка журнала СМВУ и актов ТОИР для обучения	используется в прототипе

Ни один из адаптеров к внешним системам в прототипе не подключён и не проверялся на живых данных заказчика.

10. Экономика: условие окупаемости

Реальных данных об актах ремонтов и стоимости инцидентов у нас нет, поэтому мы не обещаем сумму экономии. Вместо этого показываем, при какой точности прогноза плановый осмотр окупается, и сравниваем с измеренной точностью.

Допущения: аварийный выезд стоит 18 500 ₽, плановый осмотр — 3 200 ₽ (нормативные ставки, уточняются заказчиком). Если осмотр канала из очереди прогноза заранее устраняет проблему, то он окупается, когда доля подтверждённых проблем в очереди не ниже 3 200 / 18 500 = 17,3 %.

Вариант	Точность в топ-100 (медиана)	Относительно 17,3 %	Какую долю найденных проблем нужно успевать устранять до аварии
deployed (в демо)	22 %	выше	79 %
lgbm_weekly	38 %	выше	46 %
правило persistence	38 %	выше	46 %

Последний столбец: если не каждая найденная проблема превращается в предотвращённую аварию, окупаемость требует, чтобы доля устранённых заранее была не ниже указанной (формула: 3 200 / (18 500 × точность)). Это условие, а не прогноз экономии: доля предотвращаемых проблем определяется только на пилоте.

> **[ОГРАНИЧЕНИЕ]** Метки основаны на журнале СМВУ, а не на актах ремонтов, поэтому «найденная проблема» — это совпадение с разметкой. Прежние сценарные оценки экономии в млн рублей в год в этой редакции не используются.

11. Установка и запуск

Docker (рекомендуется)

Требуется Docker с Compose. В корне репозитория:

```
docker compose up --build
```

Интерфейс — http://localhost:8000, документация API — http://localhost:8000/docs. Данные журналов хранятся в каталоге backend/data репозитория (монтируется в контейнер). Проверка живости: GET /api/health.

Без Docker

Python 3.10+, Node.js 20+:

```
pip install -r requirements.txt
cd frontend && npm ci && npm run build && cd ..
python -m uvicorn backend.app.main:app --host 0.0.0.0 --port 8000
```

Проверка и воспроизведение

Команда	Что делает
python -m pytest tests/ -q	108 автотестов (API, калибровка, бэктест, Сценарный полигон)
python scripts/reproduce_metrics.py	пересчёт метрик модели из кэша признаков
python scripts/rolling_backtest.py	пересчёт недельного бэктеста (с ключом --extract — из исходного журнала)
python tools/heldout_eval.py	прогон отложенных наборов Сценарного полигона

Для воспроизведения метрик исходный журнал (1,5 ГБ) не нужен: используется кэш признаков; контрольные суммы данных и порядок воспроизведения — в docs/DATA_PROVENANCE_AND_REPRODUCIBILITY.md.

12. Используемые библиотеки

Серверная часть (requirements.txt), версии зафиксированы:

Библиотека	Версия	Библиотека	Версия
fastapi	0.141.1	numpy	2.2.6
uvicorn[standard]	0.53.0	pandas	2.3.3
pydantic	2.13.4	joblib	1.6.0
lightgbm	4.7.0	pytest	9.1.1
scikit-learn	1.7.2	httpx	0.28.1

Интерфейс (frontend/package.json):

Пакет	Версия	Пакет	Версия
@tailwindcss/postcss	^4.3.3	lucide-react	^1.16.0
@types/leaflet	^1.9.12	postcss	^8.5.28
@types/react	^18.3.3	react	^18.3.1
@types/react-dom	^18.3.0	react-dom	^18.3.1
@vitejs/plugin-react	^4.3.4	tailwindcss	^4.3.3
autoprefixer	^10.6.1	typescript	^5.5.3
leaflet	^1.9.4	vite	^6.4.3

Все компоненты — открытое программное обеспечение; исходный код проекта открыт и не обфусцирован.

13. Ограничения и план пилота

Ограничения текущей версии

- Метки — по журналу СМВУ, а не по актам ремонтов. Метрики показывают совпадение с этой разметкой.
- Модель в демо обучена в январе и не переобучалась; она слабее простого правила (п. 4). Еженедельное переобучение проверено бэктестом, но в демо не включено.
- Быстрый контур проверен на синтетических данных; на реальных аварийных записях Москоллектора не проверялся.
- Не подключены СМВУ, ОДС, CMMS в реальном времени; хранилища — JSON-файлы (на пилоте — PostgreSQL).
- Расчёты скорости не включают построение признаков из сырого потока.

План опытно-промышленной эксплуатации

Этап	Срок	Что делаем	Критерий перехода
0. Ретро-валидация	2–4 недели	получаем журнал ОДС и акты ТОИР за 12–24 месяца, переобучаем и проверяем на реальных исходах	ROC-AUC ≥ 0,78 на отложенной выборке; точность ≥ 18 % при полноте ≥ 65 %; отсутствие утечек данных
1. Теневой режим	1–2 месяца	читаем поток СМВУ, пишем прогнозы в отдельный журнал, диспетчеры работают как обычно	p95 задержки < 1 с; доступность ≥ 99,9 %; опережение сигнала СМВУ не менее 4 часов в ≥ 50 % случаев
2. Советчик диспетчера	3–6 месяцев	рекомендательная карточка на рабочем месте ОДС, решение остаётся за диспетчером	снижение доли ложных выездов (кластерная оценка, методика — docs/DATA_REQUEST_AND_PILOT_METHODOLOGY.md)

Требуемые от заказчика данные (журнал ОДС, акты ТОИР, тарифы, паспорта каналов) перечислены в той же методике, раздел 2.